

The single-carrier product-share gadgetizer

Construction, bookkeeping, and the statistical measurements

2026-07-26

Assumes docs/PRODUCT_SHARE_ENCODING, docs/BAND_HARDENING and docs/NONLINEAR_GADGETIZATION. This document describes the construction as it now stands, including the single-carrier decode, and presents the measurements that support it — in particular the tradeoff between the two shipped mask plans.

1 What the gadgetizer builds, and against whom

The gadgetizer rewrites a circuit A (in production, a sliced sandwich of width $2n$ wrapping the source circuit C) into a much larger equivalent G that computes the same function on its low wires when every other wire enters at 0. The security goal is that G leak nothing about A beyond its input/output behaviour, and the specific leak this line of work attacks is the **computational-progress diagonal**: the ability to tell *where in the original computation* a point of G sits.

Two execution-based instruments measure it, and they answer different questions. **hmap_affine** asks whether C 's state at prefix i is an *exact* GF(2) function of degree $\leq d$ of G 's wires at prefix j , scoring every inexact cell at 0.5; a low-error valley advancing with i is the diagonal. **hmap_stat** asks the complementary question — how well can a cheap predictor *guess* the bit — and is sensitive to bias, which the exact-span measure is structurally blind to. Both are read by the ridge measure, and on plates with unencoded ports they must be read by **interior row structure**, never by the global ρ (§??).

2 The encoding

Each logical value v_i is carried as

$$v_i = \underbrace{w_{u(i)} [\oplus w_{u'(i)}]}_{\text{one or two linear carriers}} \oplus \underbrace{\bigoplus_j \prod_l (\text{loc}[b_{jl}] \oplus a_{jl})}_{k_{\text{total}} \text{ product mask terms}} \oplus c_i, \quad (1)$$

where each mask term is a conjunction of deg band-variable literals with compile-time offsets a_{jl} , and c_i is a compile-time ledger constant. The a and c bits never appear on a wire.

2.1 Why a linear term is forced, and why one is enough

Balance obstruction. No exact reversible gadget can conditionally flip a value whose decode has unequal representation-class sizes, with any ancillae, garbage or re-encoding: exactness forces a bijection of $\{v_x = \alpha, v_t = \beta\}$ onto $\{v_x = \alpha, v_t = \beta \oplus \alpha\}$, and counting representations gives $R'(0)/R'(1) = R(0)/R(1)$ from the $\alpha = 0$ equations and $R'(0)/R'(1) = R(1)/R(0)$ from the $\alpha = 1$ equations, jointly forcing $R(0) = R(1)$. A pure two-wire product decode is 3:1, hence ungateable. Since every *balanced* two-wire decode is affine, the nonlinearity must come from wires beyond the value's own pair.

But that argument only excludes having *no* linear part. Write the decode as $D = c \oplus g(\text{sources})$ and fix the sources arbitrarily: flipping c flips D , so the two classes are exactly equal for *any* g . **One linear term discharges the obstruction completely.**

And the second linear term is free to the adversary. An affine adversary obtains every degree-1 term at no cost by XORing it into its predictor, so the second carrier never enters the piling-up product, which runs over the *nonlinear* terms alone. Writing a plan as the multiset of term degrees, the shipped $[3, 3]$ decode is really $[1, 1, 3, 3]$, and the second 1 buys nothing statistically.

2.2 Plan notation and the piling-up value

Under jointly uniform sources with variable-disjoint terms, the naive predictor (the XOR of the linear carriers) agrees with the value with probability

$$\frac{1}{2} + \frac{1}{2} \prod_{j: d_j > 1} (1 - 2^{1-d_j}), \quad (2)$$

the product running over the nonlinear terms only. This gives 0.781 for $[1, 1, 3, 3]$ and $[1, 3, 3]$ alike, 0.641 for $[1, 2, 3, 3]$, and 0.594 for $[1, 2, 2, 3]$. Section ?? records the hypothesis under which (??) is a theorem rather than a model, and why it is not yet met.

3 W: the sharing bookend and the ports

3.1 Two-carrier build

The input port emits, in order: a randomising bookend, the band fill, the W_i sharing ramp, and the mask injection ramp.

Bookend. `rand_z_gates` emits $g57$ gates whose *targets* are drawn round-robin from the aux half $[n, 2n)$ — each aux wire is a target exactly once per n -gate round — while both controls are drawn uniformly from *all* $2n$ wires. No data wire is ever a target, so the low n wires still hold x when the band fill and the ramp run. Its size is $\max(2n \lfloor \ln n \rfloor, 64)$ gates (note the floor: at $n=128$ this is 1024, not 1243).

W_i ramp. For each value, `emit_w_i_cnot` emits seven width-1 CNOTs realising the GF(2)-linear map $(a, b, c, d) \mapsto (c, d, a \oplus b, b)$ on (data, aux, share, pad). The effect is that the wire chosen as **share** receives $v \oplus z$ and the wire chosen as **pad** receives z , so $\text{share} \oplus \text{pad} = v$; simultaneously the old contents of those two wires are relocated onto the vacated data and aux wires. **share** and **pad** are drawn by rejection uniformly from all $2n$ carriers, never from the band, which is exactly why relocation bookkeeping is needed: a draw may land on a wire currently holding another value's

material. That bookkeeping is the `Slot` enum (`Data/Aux/Pair/Output`) plus `reloc`, which rewrites `dloc`, `aloc` or the matching half of `pairs`.

Decode bookend. The output port runs W_i^{-1} (the same seven CNOTs reversed, each transvection being self-inverse), borrowing a carrier when neither the share nor the pad already sits on the value’s home wire.

3.2 Single-carrier build

There is **no W_i ramp and no `rand.z` bookend at all**. The band fill is the first thing emitted; `pairs[v]` is initialised to the degenerate tuple (v, v) , so value v simply sits on wire v ; and `inject.all` — one conjunction per planned mask term, $n k_{\text{total}}$ in total — *is* the entire ramp. The value is masked in place. Exit is the mask strip followed by a routing permutation (§??).

This makes the single-carrier construction structurally *simpler* than the paired one, and it is worth being explicit that all port-side hiding then comes from the mask injection rather than from a randomised sharing.

3.3 The band fill

The **linear** fill gives each band wire its input junk XOR $\langle \alpha, x \rangle$, with α uniform on $\{0, 1\}^n$ conditioned on Hamming weight ≥ 2 (the subset is resampled until it has at least two members). The **nonlinear cascade** fill instead emits, per band wire in list order,

$$\text{band}_j = \text{junk} \oplus x_{p_j} \oplus (\text{small linear part}) \oplus \bigoplus^M (\text{two-source products}),$$

where the product sources are drawn from data wires and from *earlier* band wires — “earlier” meaning earlier *position in the wire list* passed in, not lower wire index, since after rolling that list is an arbitrary set. A transitive-support map is maintained per filled wire so that a cascade reference which would drag in this wire’s own pivot is filtered out. The cascade is what reaches high input degree with nothing wider than a two-control gate; a flat high-degree monomial would fire on a 2^{-d} fraction of inputs and behave like its linear part.

Balance. The pivot appears exactly once, linearly, and is excluded from the linear pool, from the data branch of the product draw, and transitively from the band branch. Hence each band bit is exactly balanced over uniform x for *any* choice of the nonlinear part. This is a *marginal* guarantee, one wire at a time, and §?? explains why that is weaker than the mask statistics actually need.

Mirror fill. The same fill is emitted again after the output bookend. In the paired build its target set is *every non-output wire* — the aux half plus the band, not the band alone — precisely because the emitted target set is part of the gate list, and filling only the band’s final wires would publish where the rolls left the band.

3.4 The zero-slice preblock

The preblock makes the gadget compute A *only* on the all-zero slice. Every gate is a positive conjunction whose target is a data wire and which reads exactly one slice wire, so every gate is individually dead when the slice is zero; slice controls are assigned balanced (each non-data wire is read at least once) and then shuffled. A third of the gates are plain CNOTs, and the remainder split between one and two data controls — the two-data-control shape is what keeps the disturbance from being affine in x .

The acceptance predicate is that *no nonzero slice acts as the identity on the data*: for every nonzero slice there *exists* at least one disturbed input. It is checked exhaustively when $n + \text{slice} \leq 20$ and by sampling otherwise, redrawing until satisfied.

Dimensions differ by build: the paired slice is $n + \text{band}$ (aux half plus band, total $2n + \text{band}$), the single-carrier slice is the **band alone** (total $n + \text{band}$).

4 CG: the share-native fold

A source gate is **target** $\oplus = \text{comp} \oplus \prod_i \text{lit}_i$. The fold substitutes each operand’s decode (??) by its *atom list*, walks the cartesian product, and emits each cross term as one conjunction fragment.

Atoms. For each control (w , positive): the linear atoms (*two* singleton atoms $[(c_0, \top)]$ and $[(c_1, \top)]$ in the paired build; exactly *one* under a single carrier, since emitting it twice would cancel); one atom per live mask slot, resolved to deg physical literals; and a *constant* atom (the empty vector) exactly when $\text{consts}[w] \oplus \neg\text{positive}$ — the ledger constant of w parity-merged with the control’s own polarity. This is the only place control polarity enters.

Fragments. Each cross term is the concatenation of one atom per operand. A term that is entirely empty is not emitted: it flips the target’s ledger constant instead (this is also how an arity-0 source, an X/NOT gate, becomes pure bookkeeping). A term with two polarities of one wire is identically zero and is dropped. Every surviving fragment is written to one carrier of the target — a coin flip in the paired build, the unnamed carrier under distributed sourcing, the single carrier otherwise.

Counts. The fragment count per source gate is the product over operands of their atom counts:

$$(2 + k_{\text{total}})^{\text{arity}} \quad \text{two-carrier}, \quad (1 + k_{\text{total}})^{\text{arity}} \quad \text{single-carrier},$$

at equal mask strength. Fragment *width* is set by which atoms are picked (1 for a carrier atom, deg for a mask atom, 0 for the constant atom), so it reaches $\text{arity} \cdot \text{max deg}$.

No bare X. Every constant is ledger-only: the gate complement, the all-constant cross term, the residual constant of a narrow realisation, and the slot injection/stripe parities. The accumulated constant is discharged once, in the final stripe, as the two-gate pair $\neg u$ then u over a random helper carrier — never as an X gate.

Emission order. Fragments are materialised, *shuffled*, and only then assigned targets. The odometer order is a static per-gate progress clock: consecutive fragments share atom prefixes, and the fold blocks would be legible with no execution at all.

Gate-local non-completeness. One atom per operand means a fragment reads at most one carrier of each operand and never a full decode; mask atoms resolve to band wires, which the roll keeps disjoint from carriers; and the target is never among its own controls. Under distributed sourcing this is no longer structural and is asserted at emission time instead (`debug_check_fragment`, a debug-build guarantee).

Narrow mode. `--prod-max-width` ladders every emission down to a width cap using a Barenco-style double sweep over *borrowed dirty* carriers: each borrow is visited an even number of times, so its unknown content cancels, which costs no extra wires and preserves the arbitrary-junk contract. Rungs must be exact width-2 conjunctions — a g57’s complement on a borrowed wire does not cancel. Narrow mode is built and measured but **not recommended**: it costs $6.4\times$ the gates, and the premise (that wide fragments would be inert fossils) was falsified — wide mask fragments are the *most* mobile gates in the circuit.

5 RG: re-randomisation

5.1 The paired build

The production path draws uniformly from three nonlinear g57 networks, once per inter-source-gate gap by default.

network	writes	effect
RG1 (deg 3)	all four wires of two pairs	swaps which carrier <i>pair</i> holds which value (<code>pairs.swap(i, j)</code>); the value decodes identically, off the other pair's wires
RG2 (deg 2)	all four wires	re-pairs: value i moves to $(\text{pad}_i, \text{share}_j)$ and j to $(\text{share}_i, \text{pad}_j)$
RG3	both carriers of one value	XORs the identical nonlinear bit into both carriers, so the carrier XOR — the value — is preserved

All three preserve every logical value, and all three leave the masks invariant because slots name band *variables* rather than wires (§??).

A deliberate trade worth stating plainly. The *linear* CNOT variants of these networks have gate-local non-completeness (no gate reads both carriers of a value; a bounded exhaustive search shows six CNOTs is minimal for RG1 with that property). The *nonlinear* networks that actually ship **do not**: RG1's every gate reads both carriers of one value, and RG3's two random controls may be the two carriers of a third value. Non-completeness was given up for low-degree opacity. It would be exactly backwards to describe the production RGs as non-complete.

5.2 The band roll

A roll picks a band variable and a uniformly random partner wire and swaps their contents with three transvections; the bookkeeping then re-points, so either two band variables trade wires, or a band variable and a carrier *trade roles*. Nothing about the encoding changes — the band variable keeps its value, the carrier keeps its value — only where the band lives. Carriers and band wires remain a partition, so a fold or inject target is never also a mask literal.

Each transvection is drawn from a mixed vocabulary: either a plain CNOT or the pair $t \oplus = s \wedge u$, $t \oplus = s \wedge \neg u$ over a random helper, which sums to exactly s and adds no constant. A roll is therefore 3–6 gates, not 3. The reason is a *gate-shape* census: before this change, nothing but rolls ever wrote a wire while it held a band variable, so those wires were written almost exclusively by width-1 CNOTs and remained separable even once the write *counts* matched.

5.3 Single-carrier: R1 and re-sourcing

RG2 and RG3 have **no single-carrier analogue**. There is no pair to re-pair; and XORing into a lone carrier changes the value, which could only be compensated by a term equal to the injected bit — that is, by re-creating the second carrier. *The two-carrier structure is the refresh mechanism.*

What remains covers both roles:

- **Position** — `emit_value.relocation` picks two distinct *values*, swaps their wires with a three-CNOT swap, and exchanges the bookkeeping. It deliberately makes no carrier-space assumption, because a roll can leave a value outside the carrier range.

- **Representation** — mask re-sourcing (`--prod-rsrc`) injects a fresh term and strips the old one, which genuinely changes the carrier’s bit. This makes `--prod-rsrc ≥ 1` load-bearing in single-carrier mode; it is documented but not asserted.

Because relocations and rolls permute values across the *whole* wire space, the exit emits a full permutation routing — cycle resolution over all wires, not a carrier-space permutation — so that wires $[0, n)$ hold the values and the band range holds junk, which is what makes the mirror fill safe.

6 The ledger: tracking the high-degree share elements

This is the bookkeeping that makes a moving band possible.

A slot names variables, not wires. A `ProdSlot` is a vector of factors (b, a) where b is a *band-variable id*. The single place a slot becomes physical is

$$\text{lits}(\text{loc}) : (b, a) \mapsto (\text{loc}[b], \neg a),$$

evaluated at emission time. `loc` maps variable id to current wire; it starts as the contiguous home range and changes *only* in `roll`. This indirection is exactly what makes a relocation free: the strip cancels the same product the inject added, because both resolve through the same map. Under distributed sourcing `loc` is empty and `lits` treats the factor id as a physical wire (the identity map) — so it is wrong to say literals are *always* resolved through `loc`.

Per-value state. A shared `plan` vector (the multiset of degrees, k copies of deg then k_{hi} of deg_{hi}), a per-value list of live slots, and a per-value constant. Slots are drawn against a *global* dedup set keyed on the sorted variable tuple *together with its offset pattern*, which is why the sizing bound counts $\binom{b}{d} 2^d$ rather than $\binom{b}{d}$.

Lifecycle. Draw (uniform distinct variables, sorted for canonical form, independent uniform offsets, rejected if already live, hard panic on exhaustion) $\rightarrow$ inject $\rightarrow$ optional re-source $\rightarrow$ strip. Injection and strip are the *same* routine, which is what makes a slot self-inverse up to its returned constant parity. Re-sourcing always injects the replacement *before* stripping the old term, so a value is never momentarily bare. The final strip walks slots in plan order (base terms first), then discharges any residual constant.

Invariants. Carriers and band wires partition the wire space at all times; a roll exchanges the two roles rather than duplicating them. No slot names a carrier of its own value. Every emission’s residual parity is folded into the value’s ledger constant. Slot identity is invariant under rolls.

Distributed sourcing (off by default). When mask factors live on ordinary carriers instead of a band, three further mechanisms appear: a reference count per wire; *invariant* S — at most one carrier of any value is named by live slots, which guarantees every value always has a writable carrier and prevents any fragment from seeing both carriers of a third value; one factor per owning value within a slot; and a write barrier that re-sources every slot naming a wire before anything writes it, forbidding the whole released set to the replacement draw. A 64-lane simulation of the emitted prefix lets the draw reject factor sets that are degenerate as *bits* (equal or complementary wires, identically-zero products) and replacements that duplicate a term the value already carries. Section ?? reports why this mode is not recommended.

7 Methodology: reading the plates

Three conventions are load-bearing, and each was learned by getting it wrong.

Read interior rows, not ρ . Both axes have unencoded ends: cell $(0,0)$ compares C ’s input state against G ’s input wires and reads perfectly in every build, encoded or not. On a plate with port plateaus the global ρ is that two-plateau shape ranked against noise in a dead middle, and its value is close to a coin flip. Report the count of interior rows with any prominence.

A capability control is mandatory for degree- d claims. A degree-2 adversary that cannot break a degree-2-masked build is too weak to conclude anything from. Two attempts at the degree-2 comparison in §?? were void for exactly this reason — once because the adversary was restricted to 40 of 256 band wires (it then holds a given degree-3 mask’s factors about 0.4% of the time), and once because rolls had moved the band out from under a fixed wire list. Both reported “dead” for every build, which would have been a false all-clear.

Match the sampling. The ridge statistic is a per-row maximum over columns, so plates with different column counts are not comparable; choose the stride per build to equalise them.

8 Measurements

All builds below share the source circuit and, at $n=128$, a byte-identical sandwich; only the encoding differs.

8.1 The decode grid at $n=16$

Ridge is degree-1 interior median prominence; agreement is `hmap_stat`’s interior median (a lower bound: the predictor family stops at two wires).

decode	gates	wires	ridge	agreement (peak)	theory
plain (no masks)	—	32	0.3750	—	1.000
[1, 1, 3, 3] two-carrier	4 848	44	0.0312	0.720 (0.877)	0.781
[1, 3, 3] single	2 866	28	0.0312	0.757	0.781
[1, 2, 3, 3] single	4 063	30	0.0312	0.655 (0.680)	0.641
[1, 2, 2, 3] single	4 115	30	0.0312	0.618 (0.643)	0.594

The ridge does not move: every encoded build is identically dead against the plain control’s 0.3750. The measured agreements track (??) to within a few points, which is itself a check on the arithmetic.

Note [1, 3, 3] measuring *worse* than [1, 1, 3, 3] at identical theory (0.757 vs 0.720). That is the probe-locality cost surfacing: with one carrier the single wire *is* the predictor, so no pair search is needed and the measure lands closer to the true value. One degree-2 term more than repays it.

8.2 Production, $n=128$ sliced sandwich

build	gates	wires	ridge (max)	ρ	agreement (peak)
[3, 3] two-carrier + roll	247 326	558	0.0000 (0.0547)	0.039	0.719 (0.749)
[1, 2, 3, 3] single, band 56	223 379	312	0.0000 (0.0547)	0.016	0.632 (0.660)
[1, 2, 3, 3] single, band 256	229 712	512	0.0000 (0.0430)	0.016	0.618 (0.652)
[1, 2, 2, 3] single, band 256	230 049	512	0.0000 (0.0430)	0.015	0.583 (0.600)

All four are dead at degree 1: no interior row exceeds prominence 0.10 and every argmax is pinned at the port column rather than advancing.

8.3 Wire homogeneity

A write-count census (min/median/max writes per wire, body only) asks whether any threshold separates the band from the carriers.

build	carriers	band	separable
[3, 3] two-carrier, roll 1	218/440/715	272/433/595	no
[1, 2, 3, 3] single, band 56, roll 1	452/711/1121	449/715/1026	no
[1, 2, 3, 3] single, band 256, roll 1	185/452/847	180/428/848	no
[1, 2, 2, 3] single, band 256, roll 1	210/449/1030	129/426/944	no
single, band 14, roll 0 ($n=16$)	154/236/375	10/17/21	YES
two-carrier, band 12, roll 0 ($n=16$)	80/142/216	12/19/19	YES

Two things follow. First, **rolling matters more in the single-carrier geometry, not less:** with one carrier per value each carrier absorbs every fold write, so an unwritten band wire stands out against a sharper contrast, and the roll-0 separation is cleaner than in the paired build. Rolls erase it at about +17% gates.

Second, **band 256 is the shape to ship.** At 512 wires it is *narrower* than the current 558, yet the carrier and band populations are equal in size with write distributions sitting on top of each other. That is qualitatively different from rolling a 46-wire minority: there is no small subpopulation left for a windowed census to narrow down — the residual that **BAND_HARDENING** conceded rolling could not fix. The restricted degree-3 attack cost rises from $\binom{46}{3} \approx 1.5 \cdot 10^4$ regressors to $\binom{256}{3} \approx 2.8 \cdot 10^6$.

8.4 Are the source bits good sources?

`source_stats` measures the ensemble the masks are built from: marginal bias, pairwise correlation, the degree-3 firing rate against the 1/8 it is assumed to have, identically-zero terms over all offset assignments, and an exhaustive scan for exact linear relations. Band wires and carriers both come out well-behaved marginally (no wire above the 3-sigma floor; triple-AND means 0.1253 and 0.1248 against 0.125). The instructive difference is structural:

exact wire-pair linear relations, by depth	banded build	distributed build
10 %	—	21
25 %	1	23
50 %	1	2
75 %	1	0

Carriers are *not* generic bits: a value that is identically 0 on the zero slice has two *equal* shares. Identically-zero mask terms occur only in distributed builds and only early. This is why the distributed draw now consults a live simulation.

9 The tradeoff: [1, 2, 3, 3] versus [1, 2, 2, 3]

Both plans have the same atom count, hence the same fold cost, and at $n=128$ they build to within 340 gates of each other on identical width. Their degree-1 ridges are identical and neither census

separates. They differ on two axes that pull in opposite directions.

[1, 2, 2, 3] **wins the degree-1 statistic**, exactly as (??) predicts: 0.583 median and 0.600 peak against 0.618 and 0.652, and against the shipped build’s 0.719 and 0.749.

[1, 2, 3, 3] **wins the degree wall**. Degree-2 terms are worth precisely nothing to a degree-2-capable adversary — they lie inside its span — so against that adversary [1, 2, 2, 3] is effectively [1, 3] and [1, 2, 3, 3] is [1, 3, 3]. Measured with a full-band degree-2 predictor, rolls off so the adversary can actually reach the band, and with the mandatory capability control:

build	ridge	ρ	verdict
control: degree-2 masks only	0.3750	0.999	ALIVE — the adversary is capable
[1, 2, 3, 3] single	0.0312	0.727	dead
[1, 2, 2, 3] single	0.0625	0.843	twice the residual

This is the “a single degree-3 term leaves a boundary residual, two do not” effect from the original validation, reproduced for the single-carrier decode.

Recommendation: [1, 2, 3, 3]. It beats the shipped decode on every axis at once — fewer gates, fewer wires, degree-1 and degree-2 ridges both dead against a capable adversary, and materially better statistics — while keeping two degree-3 terms. [1, 2, 2, 3] buys roughly 0.035 of degree-1 margin and spends half the degree-2 margin for it; the wall is the thing meant to hold as adversary budgets grow. [1, 2, 2, 3] remains a one-flag alternative if the threat model ever narrows to degree-1 statistical adversaries specifically.

10 Distributed sourcing: a negative result

For completeness, since the machinery is in the tree behind `--prod-src-dist`. Sourcing masks on ordinary carriers, with a write barrier replacing the freeze, is exact and costs no extra wires — and is **not recommended**. At $n=128$ it costs +63% gates (403 556 against 247 326; the bill is 79 893 forced migrations against 7 919 ordinary re-sources) *and* leaves a faint but monotonically progress-aligned ridge (interior median 0.0312, $\rho = 0.995$, perm- z 9.94) where the band build leaves none.

Eight interventions failed to move it: migration-traffic volume (a band build with churn raised to match stays dead), churn rate (a plateau at $64\times$ churn), source location (restricting to the sandwich’s keyed-junk half changes nothing), carrier routing, mask offsets, fill quality, dead terms, and duplicate terms. Dumping the fitted relations shows what the leak *is*: every leaking relation has support exactly 2, the interior count is 3 891 against the band build’s 2, and four of five sampled pairs are carriers of *different* values — a cross-value linear relation, not a value that lost its mask (a census of bare values reads 0/256 at every position). The micro-mechanism is not established.

11 What is not proved

The piling-up numbers are measurements, not theorems. Equation (??) assumes the mask’s source bits are *jointly* uniform and that a value’s terms are variable-disjoint. What the fill establishes is *marginal* balance, one wire at a time, and the accompanying test checks exactly that marginal. A mask multiplies three band wires together. Two concrete gaps: the nonlinear fill draws each wire’s pivot *with* replacement across band wires, and each wire’s exclusions cover only

its *own* pivot, so one wire’s pivot can re-enter another’s linear part. A reserved pivot block (distinct pivots, the whole pivot set excluded from every wire’s non-pivot material) makes the band exactly uniform on $\{0, 1\}^b$ for *any* nonlinear part, at zero gate cost — see `docs/BAND_INDEPENDENCE`. Until then, treat `(??)` as a model the measurements happen to track.

One-probe independence is lost in single-carrier mode. A two-carrier share is uniform *and* independent of the value, so a single probe learns nothing; a lone carrier holds $v \oplus \text{masks} \oplus \kappa$, correlated with the value at the piling-up rate. The ledger constant does not rescue this: it is a compile-time bit, identical for every input, so it flips the *sign* of that correlation rather than its magnitude, and one bit is learnable from a handful of samples. This is better against pair-searching and affine adversaries — which every heatmap here is — and worse against a strict one-probe adversary.

Agreement is not security. 0.62 agreement on a bit is a 0.12 bias, amplifiable across snapshots and across correlated cells. These are lower bounds from a bounded predictor family, and the construction has no upper bound of any kind at present.

Scope. Everything measured here is execution-based, per-instance, and single-snapshot: one (C, G) pair, one time-pair per cell, uniform inputs with the slice pinned. It says nothing about the distributional game $G(C_1)$ versus $G(C_2)$, about joint multi-snapshot leaks, about subcube-conditional adversaries, or about search/SAT-guided attacks. The static/syntactic channel is covered only by the ad-hoc censuses reported above.

12 Options

flag	env	meaning
<code>--prod-k / --prod-deg</code>	<code>PROD_K / PROD_DEG</code>	base terms and their degree
<code>--prod-k-hi / --prod-deg-hi</code>	<code>PROD_K_HI / PROD_DEG_HI</code>	tower terms and their degree
<code>--prod-single</code>	<code>PROD_SINGLE</code>	single-carrier decode (one linear term)
<code>--prod-band</code>	<code>PROD_BAND</code>	band width; 0 = auto
<code>--prod-rsrc</code>	<code>PROD_RSRC</code>	re-source moves per gap; load-bearing under <code>--prod-single</code>
<code>--prod-roll</code>	<code>PROD_ROLL</code>	band relocations per gap
<code>--prod-fill-nl</code>	<code>PROD_FILL_NL</code>	nonlinear cascade fill
<code>--prod-max-width</code>	<code>PROD_MAX_WIDTH</code>	narrow mode (not recommended)
<code>--prod-src-dist</code>	<code>PROD_SRC_DIST</code>	distributed sourcing (not recommended, §??)

The plan is the multiset built as k copies of deg then k_{hi} of deg_{hi} ; the encoding is enabled iff $k > 0$ or $k_{hi} > 0$ (the degree flags alone are inert), and `--prod-deg 1` is silently clamped to 2. The two recommended settings are

<code>[1, 2, 3, 3]</code> (recommended)	<code>--prod-single 1 --prod-k 1 --prod-deg 2 --prod-k-hi 2 --prod-deg-hi 3</code>
<code>[1, 2, 2, 3]</code>	<code>--prod-single 1 --prod-k 2 --prod-deg 2 --prod-k-hi 1 --prod-deg-hi 3</code>

both with `--prod-band 256 --prod-fill-nl 2 --prod-roll 1` at the production sandwich. Auto band width is $\max(\lceil \sqrt{4n k_{\text{total}}} \rceil, 6, \max \text{deg} + 3)$, where n is the *entry point*’s value count — the sandwich width $2n$, not the source circuit’s n . `--prod-single 0` reproduces the previous build

bit-identically.